



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Progetto di Reti Logiche

PROVA FINALE DI
INGEGNERIA INFORMATICA

Studente: **Antonio Rivero**

Matricola Studente: 982218
Codice Persona: 10778806
Professore: Gianluca Palermo
Anno accademico: 2023-24

1. Introduzione

Il progetto mira a sviluppare un modulo hardware scritto in VHDL che si interfaccia con una memoria per elaborare una sequenza di parole inserendo dei valori di credibilità.

L'obiettivo del modulo è leggere una sequenza di parole dalla memoria, completare la sequenza sostituendo valori zero con l'ultimo valore non zero letto, e aggiungere un valore di credibilità associato a ciascun valore della sequenza.

Il valore di credibilità viene inizialmente impostato a 31 ogniqualvolta il valore della sequenza è diverso da zero; viene invece diminuito progressivamente ogni volta che la sequenza incontra uno zero, con un valore minimo di zero.

1.1. Esempio

- ADD: Indirizzo di memoria della prima parola
- K: Dimensione della sequenza

Nell'esempio che si riporta di seguito consideriamo una sequenza di 11 valori ($K = 11$), partendo dall'indirizzo di memoria ADD fornito dal modulo.

Scorrendo la sequenza, la specifica richiede che:

- quando viene letto un valore diverso da zero, viene inserito nella cella successiva il valore di credibilità di 31.
- quando è presente un valore uguale a zero, viene inserito l'ultimo valore letto della sequenza e nella cella successiva viene inserito l'ultimo valore di credibilità diminuito di uno.

I valori di credibilità sono posizionati a fianco ai valori della sequenza (mostrati in grassetto).

ADD	ADD+2																	ADD+2*K			
↓		↓																↓			
128	0	64	0	0	0	0	0	0	0	0	0	0	100	0	1	0	0	5	0		
																			↓		
128	31	64	31	64	30	64	29	64	28	64	27	64	26	100	31	1	31	1	30	5	31

*in rosso i valori di credibilità
in blu i valori mancanti

Secondo la specifica, il valore di credibilità non scende mai sotto zero. Quindi, se in questo stesso esempio la sequenza di valori di memoria uguali a 0 dopo 64 fosse continuata per più di 30 occorrenze, il valore di credibilità sarebbe rimasto a zero.

...	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
																				↓
...	5	64	4	64	3	64	2	64	1	64	0	64	0	64	0	64	0	64	0	0

1.2. Interfaccia del componente

Il componente ha una interfaccia così definita:

```
component project_reti_logiche is
    port (
        i_clk : in std_logic;
        i_rst : in std_logic;
        i_start : in std_logic;
        i_add : in std_logic_vector(15 downto 0);
        i_k : in std_logic_vector(9 downto 0);

        o_done : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we : out std_logic;
        o_mem_en : out std_logic
    );
end component project_reti_logiche;
```

In particolare:

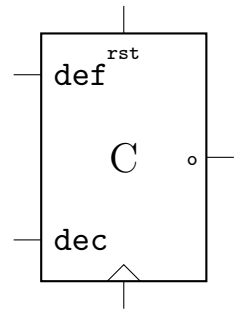
- `i_clk` è il segnale di `CLOCK` in ingresso generato dal Test Bench
- `i_rst` è il segnale di `RESET` che inizializza la macchina pronta per ricevere il primo segnale di start
- `i_start` è il segnale di start generato dal Test Bench
- `i_k` è il segnale (vettore) `K` generato dal Test Bench rappresentante la lunghezza della sequenza
- `i_add` è il segnale `ADD` generato dal Test Bench che rappresenta l'indirizzo dal quale parte la sequenza da elaborare
- `o_done` è il segnale `DONE` di uscita che comunica la fine dell'elaborazione
- `o_mem_addr` è il segnale (vettore) che arriva dalla memoria e contiene il dato in seguito a una richiesta di lettura
- `o_mem_data` è il segnale (vettore) che va verso la memoria e contiene il dato che verrà successivamente scritto
- `o_mem_en` è il segnale di `ENABLE` da dover mandare alla memoria per poter comunicare (sia in lettura che in scrittura)
- `o_mem_we` è il segnale di `WRITE ENABLE` da dover mandare alla memoria (=1) per poter scriverci. Per leggere da memoria esso deve essere 0

2. Architettura

Il design del progetto include quattro registri, arricchiti con funzioni supplementari per la gestione delle variabili. Queste ultime saranno impiegate dalla macchina a stati. Per quanto riguarda le scelte progettuali, il segnale di reset opera in modo asincrono e reimposta il valore di tutti i registri a 0. Al contrario, tutti gli altri segnali sono sincronizzati sul fronte di salita del clock.

a. Registro C

```
entity c_reg is
  port(
    decrement: in std_logic;
    clk : in std_logic;
    rst: in std_logic;
    def: in std_logic;
    output: out std_logic_vector(7 downto 0)
  );
end c_reg;
```

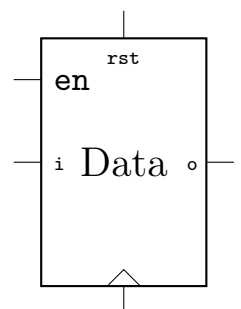


Il registro si occupa di salvare e gestire il valore di credibilità, prendendo in ingresso:

- def: riporta il valore del registro a 31
- decrement: decrementa il valore del registro di uno

b. Registro Dati

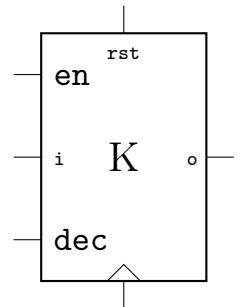
```
entity data_reg is
  port(
    input: in std_logic_vector(7 downto 0);
    clk : in std_logic;
    rst: in std_logic;
    enable: in std_logic;
    output: out std_logic_vector(7 downto 0)
  );
end data_reg;
```



Il registro ha il compito di registrare il dato della sequenza che riceve in input dalla memoria. Si tratta di un normale flip-flop di tipo D con segnale di enable.

c. Registro K

```
entity k_reg is
  port(
    input: in std_logic_vector(9 downto 0);
    enable: in std_logic;
    decrement: in std_logic;
    clk : in std_logic;
    rst: in std_logic;
    output: out std_logic_vector(9 downto 0)
  );
end k_reg;
```



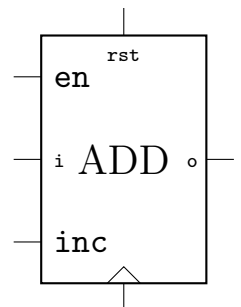
Il registro è un flip-flop di tipo D con segnale di enable.

Esso ha il compito di memorizzare la lunghezza della sequenza non appena viene attivato il segnale di `i_start`. Successivamente, riducendo progressivamente il valore memorizzato, il registro segnalerà alla macchina a stati quando la sequenza è giunta al termine.

Al registro è stato aggiunto anche un segnale di `decrement` che decrementa di uno il valore interno del registro stesso.

d. Registro ADD

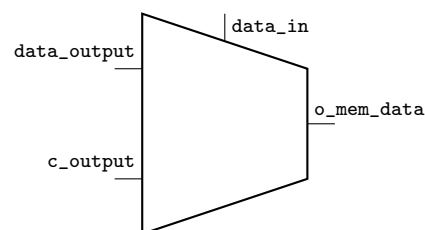
```
entity addr_reg is
  port(
    input: in std_logic_vector(15 downto 0);
    enable: in std_logic;
    increment: in std_logic;
    rst: in std_logic;
    clk : in std_logic;
    output: out std_logic_vector(15 downto 0)
  );
end addr_reg;
```



Si tratta anche in questo caso di un flip-flop di tipo D con segnale di enable. Al registro è stato aggiunto anche un segnale di `increment` che incrementa di uno il valore interno del registro. L'uscita è direttamente collegata a `o_mem_addr` per richiedere il dato dalla memoria.

e. Multiplexer

Un multiplexer prende in ingresso l'output del registro `c` e del registro dati e manda in uscita `o_mem_data` il segnale corretto a seconda del segnale di selezione `data_in` (1 -> `data_output`, 0 -> `c_output`)



In Figura 1 è rappresentato lo schema complessivo del circuito.

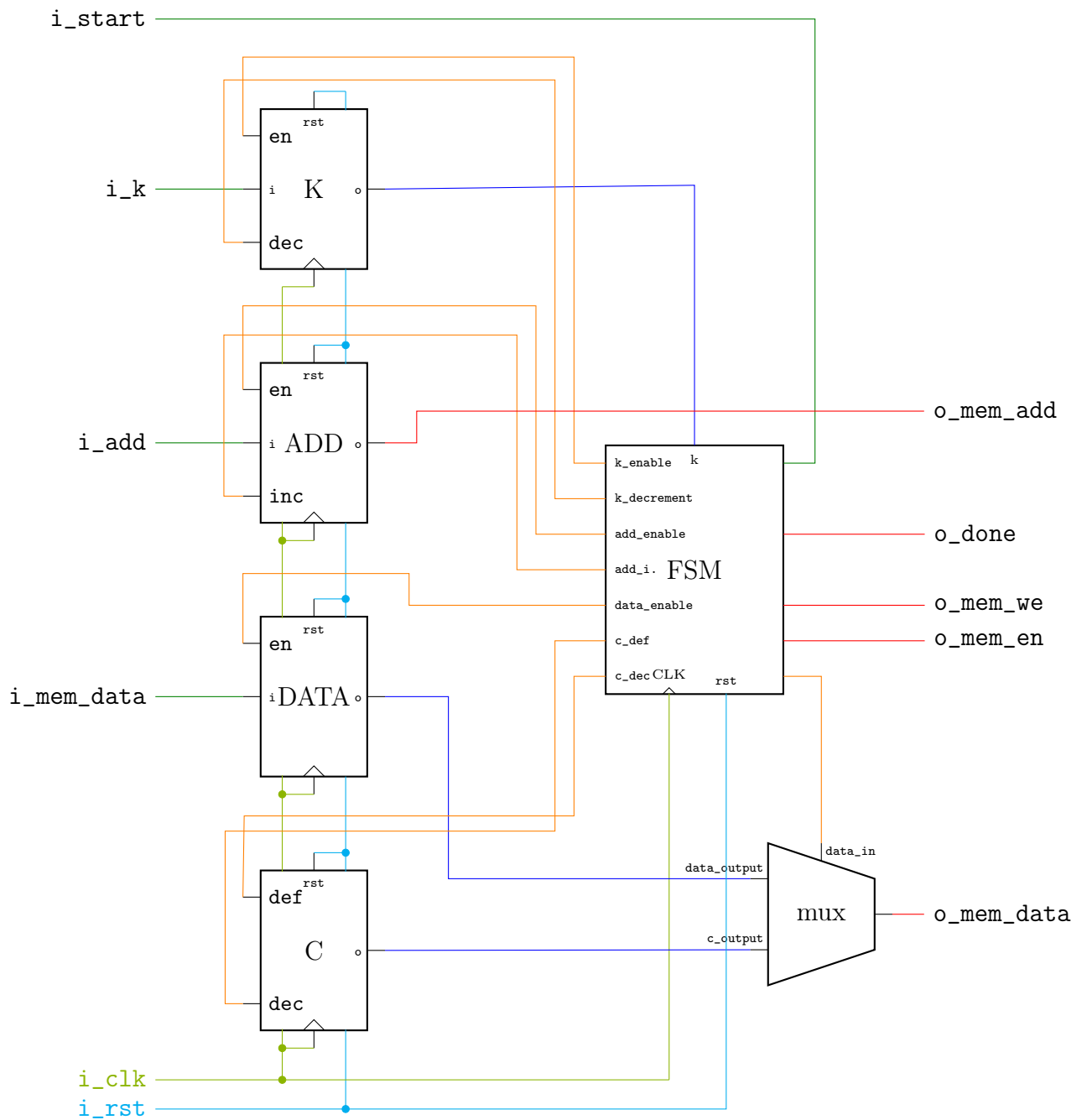


Figura 1: Schema del circuito

f. Macchina a stati

Una macchina a stati, con aggiornamento dello stato sincronizzato sul fronte di salita del clock, controlla gli input dei registri e del multiplexer (vedi Figura 2).

La macchina costruita si compone di 6 stati:

S0 - stato di reset

Lo stato iniziale della macchina è caratterizzato dall'attivazione di tutti i segnali di reset nei registri. In questo stato, viene verificato se il segnale `i_start` è stato attivato.

Se sì, i valori dei segnali `i_k` e `i_add` vengono memorizzati (`k_enable` <= '1' e `addr_enable` <= '1').

Inoltre, si verifica se `i_k` è diverso da zero, indicando che non siamo nel caso limite. Se è diverso da zero, il prossimo stato è S1. Se invece `i_k` è zero, la macchina passa allo stato S5, che rappresenta lo stadio finale.

Nel caso in cui il segnale `i_start` non sia ancora stato attivato, la macchina rimane nello stato iniziale, S0. Lo stato di reset viene raggiunto in modo asincrono ogniqualvolta il segnale di reset (`i_rst`) viene alzato.

S1 - attesa

Lo stato S1 è uno stato di attesa per consentire la lettura corretta da memoria.

(`o_mem_en` <= '1')

Il prossimo stato è S2.

S2 - controllo dato sequenza

Nello stato S2, viene verificato il valore del dato in ingresso `i_mem_data`.

Se questo valore è diverso da zero, viene portato il valore di credibilità a 31 (`c_def` <= '1') e il dato viene salvato nel registro (`data_enable` <= '1').

Se invece il dato è zero, viene decrementato il valore di credibilità nel registro (`c_decrement` <= '1') e non viene alzato il segnale di enable nel registro dati (`data_enable` <= '0').

Il prossimo stato è S3.

S3 - scrittura del dato sequenza

Nello stato S3, viene scritto il dato salvato nel registro in memoria (`data_in` <= '1'; `o_mem_en` <= '1'; `o_mem_we` <= '1'). Viene decrementato il registro k (`k_decrement` <= '1') e incrementato il registro add (`addr_increment` <= '1').

Il prossimo stato è S4.

S4 - scrittura del valore di credibilità

Nello stato S4, viene scritto valore di credibilità salvato nel registro (`data_in` <= '0'; `o_mem_en` <= '1'; `o_mem_we` <= '1';). Viene inoltre incrementato il registro add (`addr_increment` <= '1').

Si verifica il valore del registro k. Se è zero, significa che la sequenza è conclusa e la macchina passa allo stadio finale S5. Altrimenti, si ritorna allo stato S1. Lo stato S1 è necessario perché occorre attendere che il dato in ingresso `i_mem_data` venga aggiornato.

S5 - stato finale

Nello stato finale viene alzato il segnale di `o_done`. Il prossimo stato sarà S0 quando `i_start` tornerà a 0.

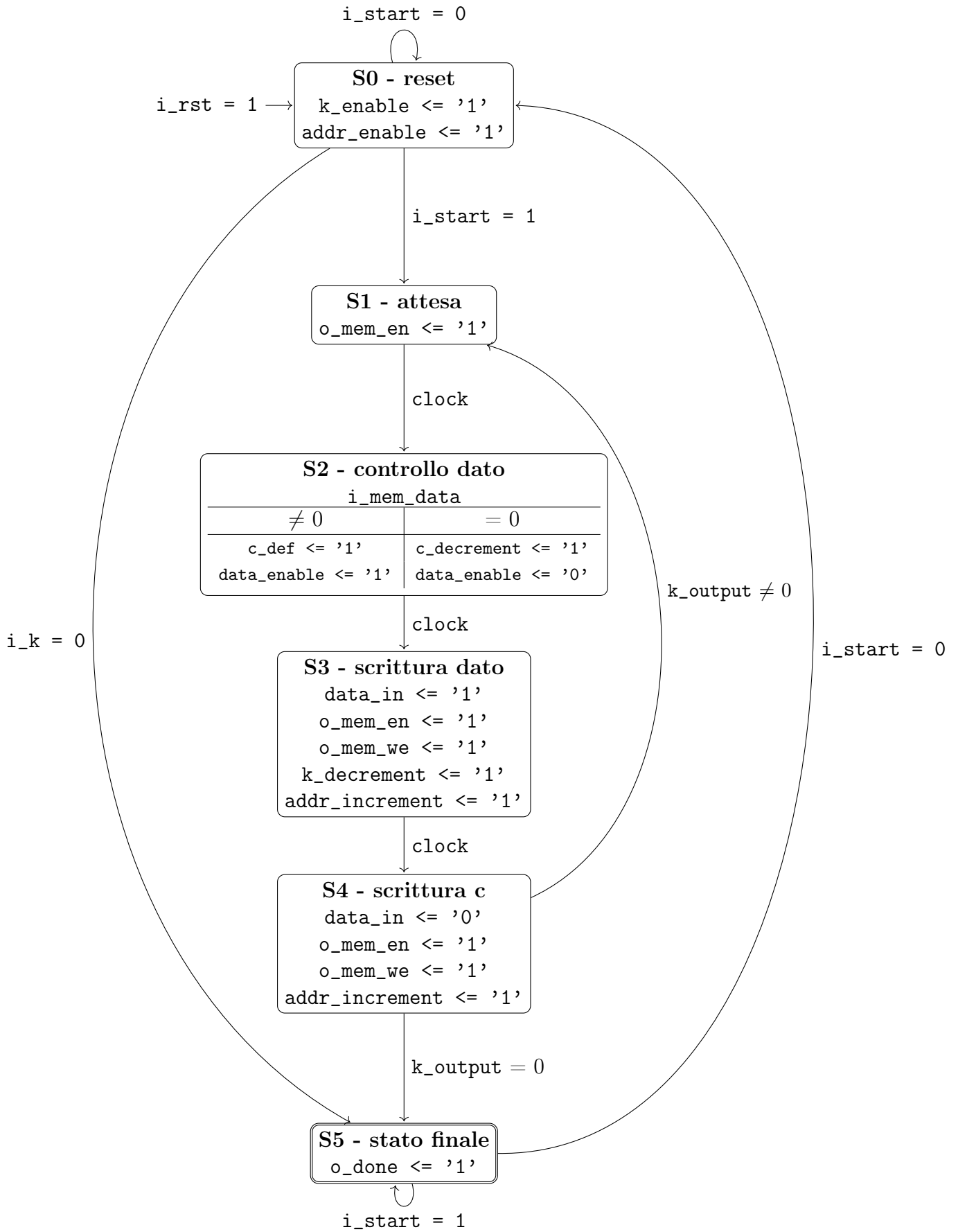


Figura 2: Macchina a stati

3. Risultati sperimentali

a. Sintesi

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	83	0	134600	0.06
LUT as Logic	83	0	134600	0.06
LUT as Memory	0	0	46200	0.00
Slice Registers	45	0	269200	0.02
Register as Flip Flop	45	0	269200	0.02
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

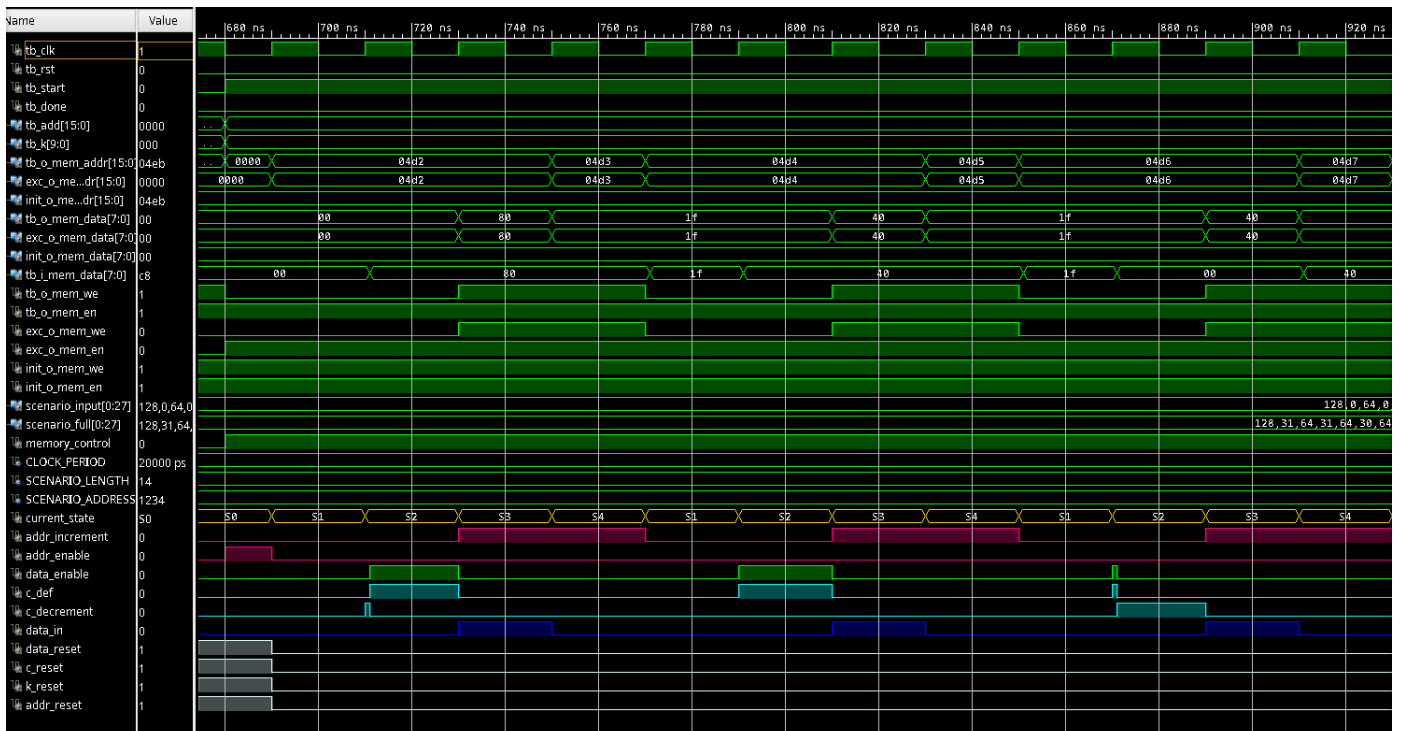
Il progetto non genera latch. I registri utilizzati sono 45, come previsto. Di questi, 8 appartengono al registro C, 8 al registro dati, 10 al registro K, e 16 al registro degli indirizzi, per un totale di 42. La macchina a stati utilizza i 3 registri rimanenti.

Slack (MET) : 16.214ns (required time - arrival time)

Il vincolo di tempo è stato ampiamente rispettato.

b. Simulazioni

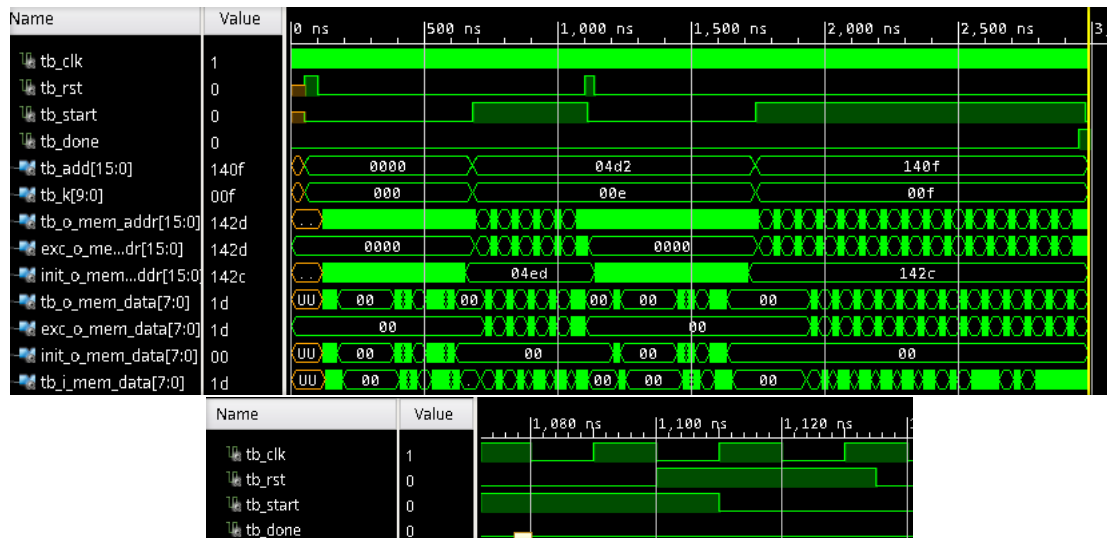
1. Test bench



Il test bench iniziale è il prototipo fornito dal docente.

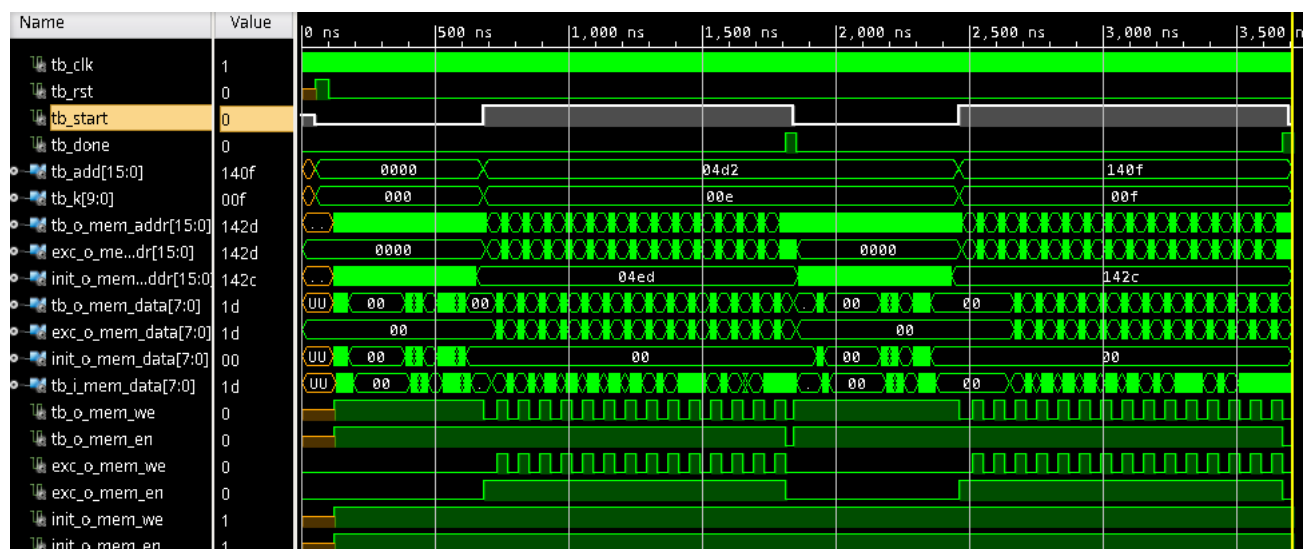
Per il test bench sono stati messi in evidenza i segnali interni della macchina e il segnale del `current_state`. Il `current_state` si aggiorna sul fronte di salita del clock. La macchina alterna correttamente tra gli stati S1 e S4. Tutti i segnali si attivano nel contesto del loro stato e il modulo funziona correttamente.

2. Test bench - reset asincrono



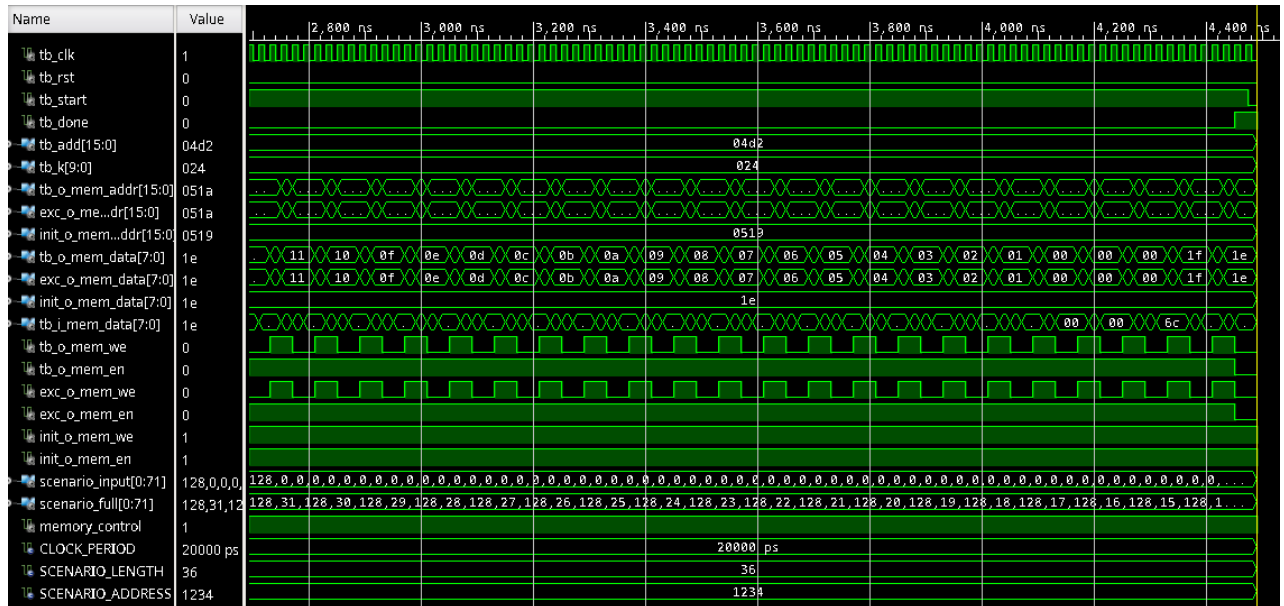
Il test verifica il caso limite del reset asincrono (attivato sul fronte di discesa), che causa un cambiamento improvviso nello scenario della simulazione.

3. Test bench - sequenze multiple



Il test verifica che il componente possa eseguire più di una sequenza.

4. Test bench - credibilità a zero



Il test verifica che il valore di credibilità non scenda sotto zero.

5. Test bench - lunghezza sequenza massima



Il test verifica che il modulo funzioni anche con il massimo valore di `i_k`.

4. Conclusioni

Il componente sintetizzato ha superato correttamente tutti i test previsti nelle tre simulazioni: *Behavioral*, *Post-Synthesis Functional* e *Post-Synthesis Timing*.

Ottimizzazioni

Il progetto è stato sottoposto a diverse ottimizzazioni, tra cui la più significativa è stata la riduzione del numero di stati.

Lo stato S2 era diviso in due stati distinti a seconda del dato in ingresso, ma questa separazione era superflua, quindi sono stati uniti in un unico stato.

Lo stato S0 è stato diviso in due stati con il secondo che avviava il processo registrando i valori di k e addr in memoria; poichè le funzionalità di questo secondo stato potevano essere svolte in contemporanea a quelle del primo, il secondo è stato eliminato in quanto superfluo.